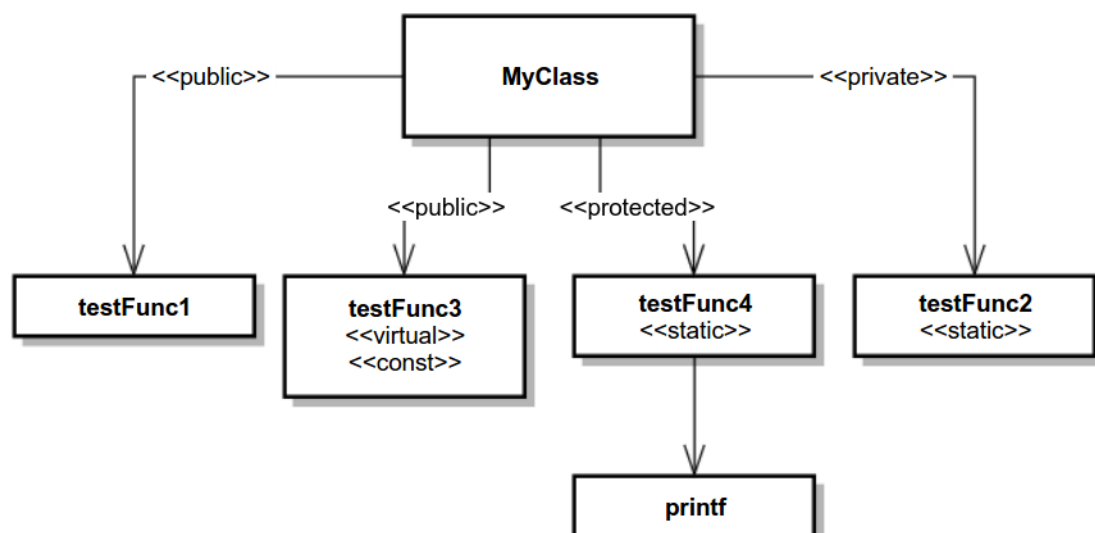


Рассмотрим приложение, способное генерировать простой код на C++ следующего вида:

```
class MyClass {  
public:  
    void testFunc1() {  
    }  
    virtual void testFunc3() const {  
    }  
  
protected:  
    static void testFunc4() {  
        printf("Hello, world!\n");  
    }  
  
private:  
    static void testFunc2() {  
    }  
};
```



Базовый класс элемента синтаксиса

Определим абстрактный класс `Unit`, представляющий некую языковую конструкцию:

```
class Unit {
public:
    using Flags = unsigned int;

public:
    virtual ~Unit() = default;

    virtual void add( const std::shared_ptr< Unit >& , Flags ) {
        throw std::runtime_error( "Not supported" );
    }

    virtual std::string compile( unsigned int level = 0 ) const =
0;

protected:

    virtual std::string generateShift( unsigned int level ) const
{
    static const auto DEFAULT_SHIFT = "    ";
    std::string result;
    for( unsigned int i = 0; i < level; ++i ) {
        result += DEFAULT_SHIFT;
    }
    return result;
}
};
```

add() - Виртуальная функция-член предназначена для добавления вложенных элементов (передача происходит через умный указатель) умный указатель `std::shared_ptr`). Также эта функция принимает параметр **Flags**. По умолчанию **add()** выбрасывает исключение.

Compile() - Функция генерирует код на C++, соответствующий содержимому элемента. Результат возвращается в виде строки **std::string**.

В качестве аргумента функция принимает параметр **level**, указывающий на уровень вложенности узла дерева. Это требуется для корректной расстановки отступов в начале строк генерируемого кода.

generateShift() - Вспомогательная функция-член всего лишь возвращает строку, состоящую из нужного числа пробелов. Результат зависит от уровня вложенности.

Реализация элемента Класс

Вот одна из возможных реализаций класса, представляющего Класс:

```
class ClassUnit : public Unit
{
public:
    enum AccessModifier {
        PUBLIC,
        PROTECTED,
        PRIVATE
    };

    static const std::vector< std::string > ACCESS_MODIFIERS;

public:
    explicit ClassUnit( const std::string& name ) : m_name( name ) {
        m_fields.resize( ACCESS_MODIFIERS.size() );
    }

    void add( const std::shared_ptr< Unit >& unit, Flags flags ) {

        int accessModifier = PRIVATE;

        if( flags < ACCESS_MODIFIERS.size() ) {
            accessModifier = flags;
        }

        m_fields[ accessModifier ].push_back( unit );
    }

    std::string compile( unsigned int level = 0 ) const
    {
        std::string result = generateShift( level ) + "class " + m_name + " {\n";

        for( size_t i = 0; i < ACCESS_MODIFIERS.size(); ++i ) {
            if( m_fields[ i ].empty() ) {
                continue;
            }

            result += ACCESS_MODIFIERS[ i ] + ":\n";
            for( const auto& f : m_fields[ i ] ) {
                result += f->compile( level + 1 );
            }
            result += "\n";
        }

        result += generateShift( level ) + "};\n";
        return result;
    }

private:
    std::string m_name;
```

```

using Fields = std::vector< std::shared_ptr< Unit > >;

std::vector< Fields > m_fields;

};
const std::vector< std::string > ClassUnit::ACCESS_MODIFIERS = { "public",
"protected", "private" };

```

Представление Функции-члена реализуем в виде следующего класса:

```

class MethodUnit : public Unit {
public:
    enum Modifier {
        STATIC      = 1,
        CONST       = 1 << 1,
        VIRTUAL     = 1 << 2
    };

public:
    MethodUnit( const std::string& name, const std::string& returnType, Flags
flags ) :
        m_name( name ), m_returnType( returnType ), m_flags( flags ) { }

    void add( const std::shared_ptr< Unit >& unit, Flags /* flags */ = 0 ) {
        m_body.push_back( unit );
    }

    std::string compile( unsigned int level = 0 ) const {
        std::string result = generateShift( level );
        if( m_flags & STATIC ) {
            result += "static ";
        } else if( m_flags & VIRTUAL ) {
            result += "virtual ";
        }
        result += m_returnType + " ";
        result += m_name + "()";
        if( m_flags & CONST ) {
            result += " const";
        }
        result += " {\n";
        for( const auto& b : m_body ) {
            result += b->compile( level + 1 );
        }
        result += generateShift( level ) + "}\n";
        return result;
    }

private:
    std::string m_name;
    std::string m_returnType;
    Flags m_flags;
    std::vector< std::shared_ptr< Unit > > m_body;
};

```

Конструктор класса **MethodUnit** принимает имя функции, тип возвращаемого значения и флаги(**name, returnType, flags**)

Для Функции-члена предусмотрено три возможных модификатора: **STATIC**, **CONST** и **VIRTUAL**.

Они определены в перечислении **Modifier** в виде битовых флагов. Их комбинацию мы и ожидаем получать в качестве третьего аргумента конструктора **flags**.

При добавлении элементов с помощью **add()** не требуется указывать дополнительные флаги. Мы просто помещаем все в конец вектора **m_body**, заполняя тело Функции.

Генерация кода в **compile()** напоминает то, что мы уже видели для элемента Класса.

Конечно, со своим специфическим синтаксисом.

Например, обратите внимание на то, как используются битовые флаги модификатора **m_flags**.

Реализация элемента "Оператор печати"

```
class PrintOperatorUnit : public Unit {
public:
    explicit PrintOperatorUnit( const std::string& text ) : m_text( text ) { }

    std::string compile( unsigned int level = 0 ) const {
        return generateShift( level ) + "printf( \"" + m_text + "\" );\n";
    }

private:
    std::string m_text;
};
```

Реализация generateProgram() – генерирует код на C++

```
std::string generateProgram() {
    ClassUnit myClass( "MyClass" );
    myClass.add(
        std::make_shared< MethodUnit >( "testFunc1", "void", 0 ),
        ClassUnit::PUBLIC
    );
    myClass.add(
        std::make_shared< MethodUnit >( "testFunc2", "void", MethodUnit::STATIC ),
        ClassUnit::PRIVATE
    );
    myClass.add(
        std::make_shared< MethodUnit >( "testFunc3", "void", MethodUnit::VIRTUAL |
MethodUnit::CONST ),
        ClassUnit::PUBLIC
    );

    auto method = std::make_shared< MethodUnit >( "testFunc4", "void",
MethodUnit::STATIC );
    method->add( std::make_shared< PrintOperatorUnit >( R"(Hello, world!\n)" ) );
    myClass.add( method, ClassUnit::PROTECTED );

    return myClass.compile();
}
```

Функция Main

```
int main() {
    std::cout << generateProgram() << std::endl;
    return 0;
}
```

Недостаток – функция generateProgram() жестко привязана к формированию кода на C++.

Задание: Сделать код расширяемым, чтобы обеспечить генерацию программ и на других языках программирования.